Rust

ENSISA 3A

Jonathan Weber

Automne 2026





Présentation du cours

Contenu du cours

Volume horaire

- Cours : 4h
- Travaux dirigés/pratiques : 24h
- Examen : 2h
- Enseignants : J. Weber

Contenu du cours

Volume horaire

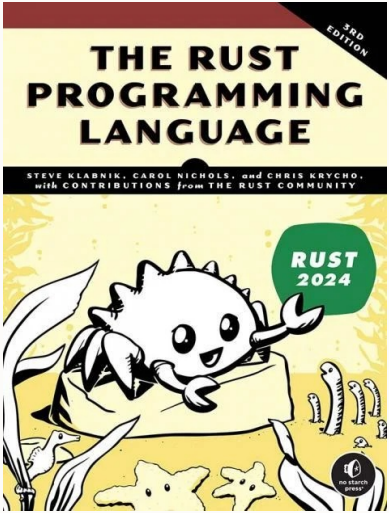
- Cours : 4h
- Travaux dirigés/pratiques : 24h
- Examen : 2h
- Enseignants : J. Weber

Évaluation

- Examen final
- Projet

Références


DONIS MARSHALL
DÉVELOPPEZ AVEC
RUST
POUR DES PROGRAMMES
SÉCURISÉS
ET PERFORMANTS
DUNOD


3RD EDITION
THE RUST PROGRAMMING LANGUAGE
STEVE KLABNIK, CAROL NICHOLS, and CHRIS KRYCHO, with CONTRIBUTIONS from THE RUST COMMUNITY
RUST 2024
NO STARCH PRESS

Pourquoi Rust ?

Pourquoi Rust a-t-il été créé ?

Le problème historique

Les langages systèmes donnent un contrôle fin sur :

- mémoire et allocation ;
- représentation des données ;
- appels système et FFI ;
- latence et performances.

Mais ce contrôle rend possibles : use-after-free, double free, data races, invalidations de pointeurs…

L'objectif Rust

Conserver le niveau de contrôle de C/C++ tout en déplaçant un maximum de vérifications vers la compilation :

- **sûreté mémoire** sans GC ;
- **concurrence** plus sûre ;
- abstractions haut niveau à coût prévisible ;
- interopérabilité avec C.

Contrôle bas niveau + garanties statiques

Pourquoi la sûreté mémoire est-elle un problème ?

Use-after-free

Une zone mémoire est utilisée après avoir été libérée.
Conséquences : crash, données corrompues, faille de sécurité.

Pourquoi la sûreté mémoire est-elle un problème ?

Use-after-free

Une zone mémoire est utilisée après avoir été libérée.
Conséquences : crash, données corrompues, faille de sécurité.

Double free

La même allocation est libérée deux fois.
Peut corrompre les structures internes de l'allocateur.

Pourquoi la sûreté mémoire est-elle un problème ?

Use-after-free

Une zone mémoire est utilisée après avoir été libérée.
Conséquences : crash, données corrompues, faille de sécurité.

Double free

La même allocation est libérée deux fois.
Peut corrompre les structures internes de l'allocateur.

Invalidation de pointeurs

Une opération déplace une allocation mais un ancien pointeur continue à être utilisé.

Pourquoi la sûreté mémoire est-elle un problème ?

Use-after-free

Une zone mémoire est utilisée après avoir été libérée.
Conséquences : crash, données corrompues, faille de sécurité.

Double free

La même allocation est libérée deux fois.
Peut corrompre les structures internes de l'allocateur.

Invalidation de pointeurs

Une opération déplace une allocation mais un ancien pointeur continue à être utilisé.

Data race

Deux threads accèdent simultanément à une donnée, dont au moins un en écriture, sans synchronisation.
Résultat non déterministe et, en C/C++, comportement indéfini.

Pourquoi la sûreté mémoire est-elle un problème ?

Use-after-free

Une zone mémoire est utilisée après avoir été libérée.
Conséquences : crash, données corrompues, faille de sécurité.

Double free

La même allocation est libérée deux fois.
Peut corrompre les structures internes de l'allocateur.

Invalidation de pointeurs

Une opération déplace une allocation mais un ancien pointeur continue à être utilisé.

Data race

Deux threads accèdent simultanément à une donnée, dont au moins un en écriture, sans synchronisation.
Résultat non déterministe et, en C/C++, comportement indéfini.

Pourquoi est-ce important ?

Ces bugs sont souvent **rares, non déterministes et difficiles à reproduire**. Ils peuvent provoquer des **crashs, corruptions de données et vulnérabilités de sécurité**.

Rust essaie de transformer une grande partie de ces erreurs d'exécution en erreurs de compilation.

La rupture conceptuelle : la mémoire fait partie du type

C / C++

Qui possède ?

Convention + discipline

`malloc/free`
RAII
smart pointers

La question **« qui peut utiliser cette valeur, et comment ? »** devient une question de compilation.

La rupture conceptuelle : la mémoire fait partie du type

C / C++	Java / Python
Qui possède ?	**Qui possède ?**
Convention + discipline	Le runtime arbitre
`malloc/free` RAII smart pointers	GC références partagées

La question **« qui peut utiliser cette valeur, et comment ? »** devient une question de compilation.

La rupture conceptuelle : la mémoire fait partie du type

C / C++	Java / Python	Rust
Qui possède ?	**Qui possède ?**	**Qui possède ?**
Convention + discipline	Le runtime arbitre	Le programme le dit
`malloc/free` RAII smart pointers	GC références partagées	ownership borrowing lifetimes

La question **« qui peut utiliser cette valeur, et comment ? »** devient une question de compilation.

Ownership : qui possède la donnée ?

Idée centrale

En Rust, chaque valeur a un **propriétaire unique** à un instant donné. Le propriétaire est responsable de la durée de vie de cette valeur.

Ownership : qui possède la donnée ?

Idée centrale

En Rust, chaque valeur a un **propriétaire unique** à un instant donné. Le propriétaire est responsable de la durée de vie de cette valeur.

Ce que cela apporte

- La libération des ressources est **déterministe**
- Pas besoin de garbage collector
- La responsabilité mémoire est explicite
- Les transferts de propriété sont vérifiés à la compilation

Ownership : qui possède la donnée ?

Idée centrale

En Rust, chaque valeur a un **propriétaire unique** à un instant donné. Le propriétaire est responsable de la durée de vie de cette valeur.

Ce que cela apporte

- La libération des ressources est **déterministe**
- Pas besoin de garbage collector
- La responsabilité mémoire est explicite
- Les transferts de propriété sont vérifiés à la compilation

Ce que Rust cherche à éviter

- double free
- use-after-free
- références vers des données déjà détruites
- ambiguïté sur “qui doit libérer quoi”

Ownership : qui possède la donnée ?

Idée centrale

En Rust, chaque valeur a un **propriétaire unique** à un instant donné. Le propriétaire est responsable de la durée de vie de cette valeur.

Ce que cela apporte

- La libération des ressources est **déterministe**
- Pas besoin de garbage collector
- La responsabilité mémoire est explicite
- Les transferts de propriété sont vérifiés à la compilation

Ce que Rust cherche à éviter

- double free
- use-after-free
- références vers des données déjà détruites
- ambiguïté sur “qui doit libérer quoi”

Modèle mental

Une ressource → un responsable clairement identifié

Ownership : qui possède la donnée ?

Idée centrale

En Rust, chaque valeur a un **propriétaire unique** à un instant donné. Le propriétaire est responsable de la durée de vie de cette valeur.

Ce que cela apporte

- La libération des ressources est **déterministe**
- Pas besoin de garbage collector
- La responsabilité mémoire est explicite
- Les transferts de propriété sont vérifiés à la compilation

Ce que Rust cherche à éviter

- double free
- use-after-free
- références vers des données déjà détruites
- ambiguïté sur “qui doit libérer quoi”

Modèle mental

Une ressource → un responsable clairement identifié

L’ownership transforme la gestion mémoire en propriété du système de types.

Borrowing : utiliser une donnée sans la posséder

Principe

Une fonction/partie du programme peut **emprunter** une valeur sans en devenir propriétaire.

Borrowing : utiliser une donnée sans la posséder

Principe

Une fonction/partie du programme peut **emprunter** une valeur sans en devenir propriétaire.

Emprunt en lecture

- plusieurs lecteurs possibles
- la donnée ne peut pas être modifiée via ces références

Emprunt mutable

- un seul accès mutable actif
- accès exclusif pendant la mutation

Borrowing : utiliser une donnée sans la posséder

Principe

Une fonction/partie du programme peut **emprunter** une valeur sans en devenir propriétaire.

Emprunt en lecture

- plusieurs lecteurs possibles
- la donnée ne peut pas être modifiée via ces références

Emprunt mutable

- un seul accès mutable actif
- accès exclusif pendant la mutation

À un instant donné :

plusieurs lecteurs

OU

un seul écrivain

Cette règle empêche les accès concurrents incohérents sur la même donnée.

Borrowing : utiliser une donnée sans la posséder

Principe

Une fonction/partie du programme peut **emprunter** une valeur sans en devenir propriétaire.

Emprunt en lecture

- plusieurs lecteurs possibles
- la donnée ne peut pas être modifiée via ces références

Emprunt mutable

- un seul accès mutable actif
- accès exclusif pendant la mutation

À un instant donné :

plusieurs lecteurs

OU

un seul écrivain

Cette règle empêche les accès concurrents incohérents sur la même donnée.

Avantage

Le compilateur peut empêcher avant exécution :

- certaines invalidations de pointeurs
- certaines modifications concurrentes incohérentes
- de nombreuses data races

Borrowing : utiliser une donnée sans la posséder

Principe

Une fonction/partie du programme peut **emprunter** une valeur sans en devenir propriétaire.

Emprunt en lecture

- plusieurs lecteurs possibles
- la donnée ne peut pas être modifiée via ces références

Emprunt mutable

- un seul accès mutable actif
- accès exclusif pendant la mutation

À un instant donné :

plusieurs lecteurs

OU

un seul écrivain

Cette règle empêche les accès concurrents incohérents sur la même donnée.

Avantage

Le compilateur peut empêcher avant exécution :

- certaines invalidations de pointeurs
- certaines modifications concurrentes incohérentes
- de nombreuses data races

Borrowing = partager l'accès sans perdre le contrôle sur la ressource.

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Rôle des lifetimes

Les lifetimes permettent au compilateur de vérifier les relations entre :

- la durée de vie d'une valeur
- la durée de vie de ses références
- la durée pendant laquelle ces références sont utilisées

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Rôle des lifetimes

Les lifetimes permettent au compilateur de vérifier les relations entre :

- la durée de vie d'une valeur
- la durée de vie de ses références
- la durée pendant laquelle ces références sont utilisées

Ce qu'elles empêchent

- dangling references
- retour d'une référence vers une donnée locale détruite
- utilisation d'une référence après destruction de sa cible

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Rôle des lifetimes

Les lifetimes permettent au compilateur de vérifier les relations entre :

- la durée de vie d'une valeur
- la durée de vie de ses références
- la durée pendant laquelle ces références sont utilisées

Ce qu'elles empêchent

- dangling references
- retour d'une référence vers une donnée locale détruite
- utilisation d'une référence après destruction de sa cible

Point important

Une lifetime ne signifie pas : "faire vivre une variable plus longtemps" Elle décrit une **relation de validité entre des références**.

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Rôle des lifetimes

Les lifetimes permettent au compilateur de vérifier les relations entre :

- la durée de vie d'une valeur
- la durée de vie de ses références
- la durée pendant laquelle ces références sont utilisées

Ce qu'elles empêchent

- dangling references
- retour d'une référence vers une donnée locale détruite
- utilisation d'une référence après destruction de sa cible

Point important

Une lifetime ne signifie pas : "faire vivre une variable plus longtemps" Elle décrit une **relation de validité entre des références**.

Résultat

Une référence valide ne peut jamais survivre à la donnée qu'elle référence.

Lifetimes : garantir la validité des références

Le problème

Une référence n'est sûre que si la donnée qu'elle désigne **existe encore pendant toute son utilisation**.

Rôle des lifetimes

Les lifetimes permettent au compilateur de vérifier les relations entre :

- la durée de vie d'une valeur
- la durée de vie de ses références
- la durée pendant laquelle ces références sont utilisées

Ce qu'elles empêchent

- dangling references
- retour d'une référence vers une donnée locale détruite
- utilisation d'une référence après destruction de sa cible

Point important

Une lifetime ne signifie pas : "faire vivre une variable plus longtemps" Elle décrit une **relation de validité entre des références**.

Résultat

Une référence valide ne peut jamais survivre à la donnée qu'elle référence.

Rust vérifie cette propriété à la compilation, sans garbage collector.

Les grosses différences à connaître avant la syntaxe

- **pas de garbage collector obligatoire** ;
- ownership, borrowing, lifetimes ;
- mutabilité explicite ;
- `enum` algébriques + pattern matching ;
- `Option` plutôt qu'un null implicite ;
- `Result` plutôt qu'un mécanisme d'exceptions pour les erreurs récupérables ;
- **traits** plutôt qu'héritage de classes ;
- généricité monomorphisée ou dispatch dynamique explicite ;
- erreurs de concurrence repoussées vers le type system ;
- `unsafe` localise les preuves que le compilateur ne peut pas faire ;
- Cargo standardise build, dépendances, tests, docs et publication.

À retenir

Rust ne remplace pas une fonctionnalité de C++ : il change le **contrat entre le programmeur et le compilateur**.

Rust : Vrai langage ou mode ?

Une histoire courte, mais déjà plusieurs époques



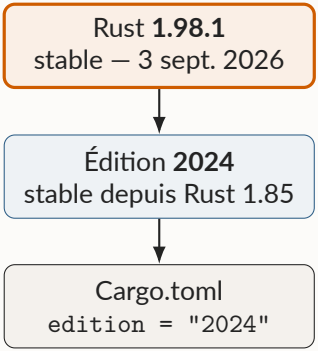
À retenir

Rust n'est plus un langage « expérimental de Mozilla » : le projet est aujourd'hui gouverné par sa communauté et la Rust Foundation, avec une forte exigence de compatibilité depuis 1.0.

Rust 2026 : quelle version ?

En septembre 2026

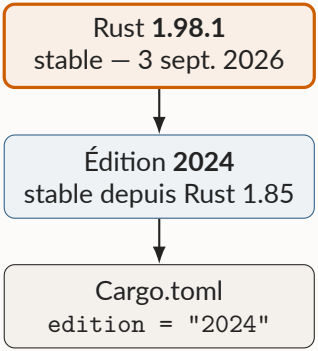
- compilateur stable : **Rust 1.98.1** ;
- édition courante du langage : **Rust 2024** ;
- cadence de publication : environ ***6* semaines** ;
- les éditions sont compatibles entre crates : elles ne remplacent pas les versions du compilateur.



Rust 2026 : quelle version ?

En septembre 2026

- compilateur stable : **Rust 1.98.1** ;
- édition courante du langage : **Rust 2024** ;
- cadence de publication : environ **6 semaines** ;
- les éditions sont compatibles entre crates : elles ne remplacent pas les versions du compilateur.



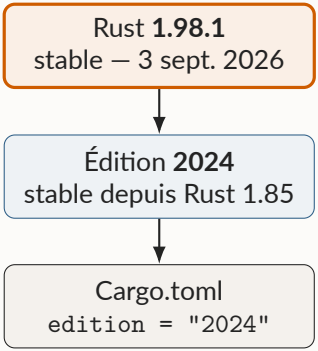
À retenir

« Rust 2026 » signifie ici **le Rust actuel utilisé en 2026**, pas une hypothétique édition 2026.

Rust 2026 : quelle version ?

En septembre 2026

- compilateur stable : **Rust 1.98.1** ;
- édition courante du langage : **Rust 2024** ;
- cadence de publication : environ **6 semaines** ;
- les éditions sont compatibles entre crates : elles ne remplacent pas les versions du compilateur.



À retenir

« Rust 2026 » signifie ici **le Rust actuel utilisé en 2026**, pas une hypothétique édition 2026.

Différence release/edition : source fréquente de confusion

Rust 1.98.1 : pourquoi une release « .1 » ?

Rust 1.98.0 — 20 août 2026

Release fonctionnelle de la série 1.98 : évolutions du langage, de la bibliothèque, des plateformes et des outils.

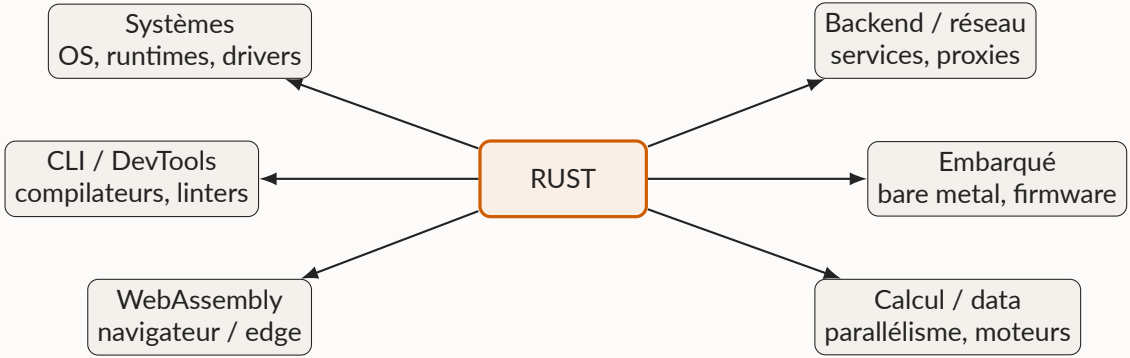
Rust 1.98.1 — 3 septembre 2026

Point release de sûreté/correction : rustc corrige une **mauvaise génération de vtable de trait object** qui pouvait placer un pointeur nul à la place d'un pointeur de fonction et conduire à de l'undefined behavior.

À retenir

Une **version du compilateur** évolue toutes les 6 semaines et peut recevoir des point releases ; une **édition** (ici 2024) fixe surtout des règles de syntaxe/migration compatibles entre crates.

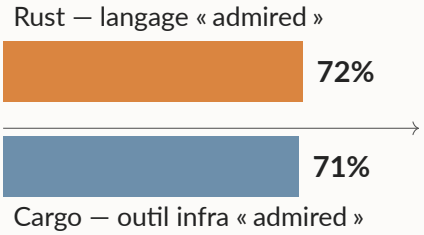
Que peut-on construire en Rust ?



Le positionnement utile

Rust est particulièrement intéressant quand **performance, sûreté, contrôle des ressources et concurrence** comptent simultanément.

Popularité : attention à ce que mesure un sondage



Source : Stack Overflow Developer Survey 2025.

- En 2025, Rust est de nouveau le langage le plus *admired* du sondage Stack Overflow : 72% des utilisateurs déclarants souhaitent continuer à l'utiliser.
- Cargo est lui aussi le premier outil « cloud development / infrastructure » sur cette métrique (71%).
- Le State of Rust Survey 2025 a recueilli **7 156 réponses complètes**.

Ne pas confondre

« Très apprécié par ses utilisateurs » **n'est pas** « langage le plus utilisé du marché ».

Adoption industrielle : le cas Android

Pourquoi Google introduit Rust

Android adopte progressivement des langages memory-safe pour le **nouveau code** et les composants sensibles : OS, firmware, services bas niveau.

Données publiées par Google en 2025

Sur le périmètre Android mesuré par Google :

- vulnérabilités de sûreté mémoire passées sous 20% du total ;
- densité de vulnérabilités mémoire annoncée comme très inférieure dans le code Rust que dans le code C/C++ ;
- moins de rollbacks et moins de temps de revue pour les changements Rust.

À retenir

Le message industriel n'est pas « tout réécrire » : c'est souvent **introduire Rust là où le nouveau code bas niveau coûte cher à sécuriser**.

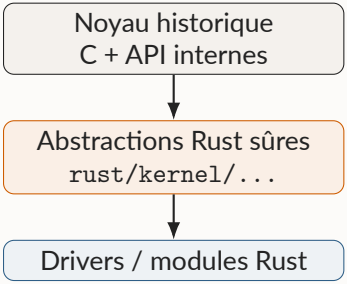
Source : Google Online Security Blog, « Rust in Android : move fast and fix things », 13/11/2025.

Rust et le noyau Linux : de l'expérience à l'infrastructure

2022 : entrée en mainline

Le support Rust est fusionné dans **Linux 6.1**. L'objectif initial est explicitement expérimental : construire les abstractions nécessaires et évaluer le langage dans le noyau.

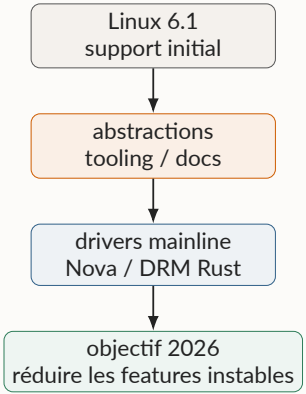
- activation via `CONFIG_RUST` ;
- environnement `no_std` ;
- intégration à la documentation et aux outils du kernel ;
- interfaces sûres autour de primitives C existantes.



Source : Linux kernel documentation — Rust ; support fusionné dans Linux 6.1.

Rust dans Linux en 2026 : où en est-on ?

- Le projet Rust for Linux décrit désormais Rust comme une **partie intégrante du noyau Linux**, avec des drivers mainline et du tooling dédié.
- La documentation des mainteneurs liste des composants GPU Rust, notamment **Nova** pour NVIDIA, ainsi qu'une infrastructure DRM Rust.
- Le support continue cependant d'évoluer : architectures, abstractions et dépendance à certaines features de langage doivent encore progresser.



Nuance importante

« Accepté dans le noyau » ne signifie pas « tout le noyau va être réécrit en Rust ». C et Rust coexistent, et l'adoption se fait subsystem par subsystem.

Source : Rust Project Goals 2026 — Rust for Linux ; Linux kernel MAINTAINERS / Rust docs, consultés en sept. 2026.

Pourquoi Rust est intéressant pédagogiquement

Il force à expliciter

- qui possède une ressource ;
- qui peut la muter ;
- combien de temps une référence est valide ;
- quelles erreurs sont récupérables ;
- quel partage entre threads est réellement sûr.

Pour vous

Vous connaissez déjà C, C++, Java, F#, Python, JavaScript, …Rust devient alors un excellent **langage de comparaison** :

- RAII sans héritage de classes ;
- ADT + pattern matching au niveau système ;
- généricité sans GC ;
- concurrence gouvernée par le type system.

À retenir

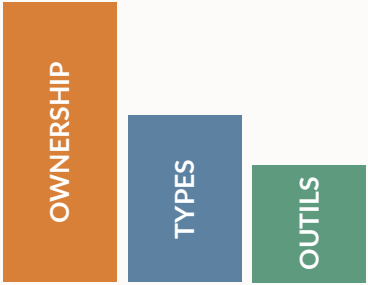
Le vrai sujet du cours : **apprendre à lire les contraintes comme une description de l'architecture du programme**.

L'objectif du cours

Vous savez déjà programmer.

Nous allons surtout étudier les endroits où **Rust refuse vos habitudes**.

- syntaxe : rapide ;
- types : structurants ;
- ownership : central ;
- abstractions : sans perdre le contrôle ;
- concurrence : conséquence du modèle ;
- unsafe : frontière explicite.



Rust : L’écosystème

La toolchain Rust : plusieurs outils, un seul écosystème

Rust ne se résume pas à un compilateur

La toolchain Rust : plusieurs outils, un seul écosystème

Rust ne se résume pas à un compilateur

Installation

`rustup`
Toolchains
Versions
Targets
Components

La toolchain Rust : plusieurs outils, un seul écosystème

Rust ne se résume pas à un compilateur

Installation
`rustup`
Toolchains
Versions
Targets
Components

Développement
`cargo`
Build
Dépendances
Tests
Documentation

La toolchain Rust : plusieurs outils, un seul écosystème

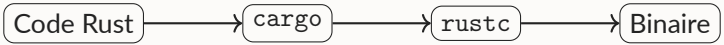
Rust ne se résume pas à un compilateur

Installation	Développement	Qualité / IDE
`rustup`	`cargo`	`rustfmt`
Toolchains	Build	`clippy`
Versions	Dépendances	`rust-analyzer`
Targets	Tests	Formatage
Components	Documentation	Analyse statique

La toolchain Rust : plusieurs outils, un seul écosystème

Rust ne se résume pas à un compilateur

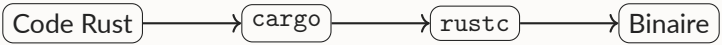
Installation	Développement	Qualité / IDE
`rustup`	`cargo`	`rustfmt`
Toolchains	Build	`clippy`
Versions	Dépendances	`rust-analyzer`
Targets	Tests	Formatage
Components	Documentation	Analyse statique



La toolchain Rust : plusieurs outils, un seul écosystème

Rust ne se résume pas à un compilateur

Installation	Développement	Qualité / IDE
`rustup`	`cargo`	`rustfmt`
Toolchains	Build	`clippy`
Versions	Dépendances	`rust-analyzer`
Targets	Tests	Formatage
Components	Documentation	Analyse statique



Une particularité importante de Rust

L'écosystème officiel fournit une chaîne d'outils très intégrée : **installation, compilation, dépendances, tests, formatage, analyse statique et support IDE**.

rustup : gérer les installations de Rust

Rôle

`rustup` est le **gestionnaire de toolchains Rust**.
Il permet d'installer et de sélectionner différentes versions du compilateur et des outils associés.

rustup : gérer les installations de Rust

Rôle

`rustup` est le **gestionnaire de toolchains Rust**.
Il permet d'installer et de sélectionner différentes versions du compilateur et des outils associés.

Une toolchain contient notamment

- `rustc`
- `cargo`
- la bibliothèque standard
- éventuellement `rustfmt`
- éventuellement `clippy`
- d'autres components

rustup : gérer les installations de Rust

Rôle

`rustup` est le **gestionnaire de toolchains Rust**.
Il permet d'installer et de sélectionner différentes versions du compilateur et des outils associés.

Une toolchain contient notamment

- `rustc`
- `cargo`
- la bibliothèque standard
- éventuellement `rustfmt`
- éventuellement `clippy`
- d'autres components

Plusieurs toolchains simultanément

```
stable
beta
nightly
1.98.1
```

Chaque projet peut utiliser une toolchain différente.

rustup : gérer les installations de Rust

Rôle

`rustup` est le **gestionnaire de toolchains Rust**.
Il permet d'installer et de sélectionner différentes versions du compilateur et des outils associés.

Une toolchain contient notamment

- `rustc`
- `cargo`
- la bibliothèque standard
- éventuellement `rustfmt`
- éventuellement `clippy`
- d'autres components

Plusieurs toolchains simultanément

```
stable
beta
nightly
1.98.1
```

Chaque projet peut utiliser une toolchain différente.

Commandes importantes

```
rustup show   rustup update   rustup toolchain list   rustup default stable
```

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

beta

Préparation de la prochaine version stable.

```
beta
```

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

beta

Préparation de la prochaine version stable.

```
beta
```

nightly

Version quotidienne. Permet notamment l'accès à certaines fonctionnalités expérimentales.

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

beta

Préparation de la prochaine version stable.

```
beta
```

nightly

Version quotidienne. Permet notamment l'accès à certaines fonctionnalités expérimentales.

Installer une version particulière

```
rustup toolchain install 1.98.1
```

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

beta

Préparation de la prochaine version stable.

```
beta
```

nightly

Version quotidienne. Permet notamment l'accès à certaines fonctionnalités expérimentales.

Installer une version particulière

```
rustup toolchain install 1.98.1
```

Utiliser ponctuellement une autre toolchain

```
cargo +1.98.1 build
cargo +nightly build
rustc +1.98.1 --version
```

rustup : stable, beta, nightly

stable

Version destinée à la production.

```
stable
```

beta

Préparation de la prochaine version stable.

```
beta
```

nightly

Version quotidienne. Permet notamment l'accès à certaines fonctionnalités expérimentales.

Installer une version particulière

```
rustup toolchain install 1.98.1
```

Utiliser ponctuellement une autre toolchain

```
cargo +1.98.1 build
cargo +nightly build
rustc +1.98.1 --version
```

Intérêt

Un projet peut être compilé avec une version **reproductible et explicitement choisie**,

rustup : components et targets

Component

Un outil ou élément appartenant à une toolchain.

```
rustup component list
rustup component add rustfmt
rustup component add clippy
rustup component add rust-analyzer
rustup component add rust-src
```

rustup : components et targets

Component

Un outil ou élément appartenant à une toolchain.

```
rustup component list
rustup component add rustfmt
rustup component add clippy
rustup component add rust-analyzer
rustup component add rust-src
```

Target

Une plateforme pour laquelle le compilateur peut générer du code.

```
rustup target list
rustup target add aarch64-unknown-linux-gnu
```

rustup : components et targets

Component

Un outil ou élément appartenant à une toolchain.

```
rustup component list
rustup component add rustfmt
rustup component add clippy
rustup component add rust-analyzer
rustup component add rust-src
```

Target

Une plateforme pour laquelle le compilateur peut générer du code.

```
rustup target list
rustup target add aarch64-unknown-linux-gnu
```

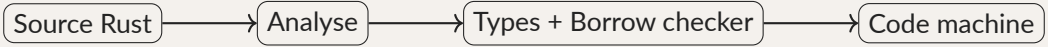
Conséquence

`rustup` simplifie fortement le **cross-compiling** et l'installation des bibliothèques standard pour d'autres plateformes.

rustc : le compilateur Rust

Rôle

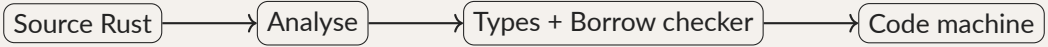
`rustc` transforme un **crate Rust** en bibliothèque ou en programme exécutable.



rustc : le compilateur Rust

Rôle

`rustc` transforme un **crate Rust** en bibliothèque ou en programme exécutable.



Compilation directe

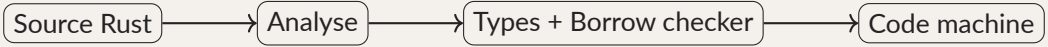
```
rustc hello.rs
```

Cela produit directement un exécutable.

rustc : le compilateur Rust

Rôle

`rustc` transforme un **crate Rust** en bibliothèque ou en programme exécutable.



Compilation directe

```
rustc hello.rs
```

Cela produit directement un exécutable.

En pratique

On invoque rarement `rustc` directement.
Cargo construit les commandes rustc à notre place.

rustc : quelle est l'unité de compilation ?

En C / C++

Chaque fichier source est généralement compilé séparément.

$$\texttt{a.cpp} \rightarrow a.o \quad \texttt{b.cpp} \rightarrow b.o$$

puis édition de liens.

rustc : quelle est l'unité de compilation ?

En C / C++

Chaque fichier source est généralement compilé séparément.

$$\texttt{a.cpp} \rightarrow a.o \quad \texttt{b.cpp} \rightarrow b.o$$

puis édition de liens.

En Rust

L'unité de compilation logique est le **crate**.

$$\boxed{\text{crate root}} \longrightarrow \boxed{\text{modules}} \longrightarrow \boxed{\text{crate compilé}}$$

Le compilateur part typiquement de :

`src/main.rs` ou `src/lib.rs`

et découvre les modules à partir de la structure du crate.

rustc : quelle est l'unité de compilation ?

En C / C++

Chaque fichier source est généralement compilé séparément.

$$\texttt{a.cpp} \to a.o \quad \texttt{b.cpp} \to b.o$$

puis édition de liens.

En Rust

L'unité de compilation logique est le **crate**.

$$\boxed{\text{crate root}} \longrightarrow \boxed{\text{modules}} \longrightarrow \boxed{\text{crate compilé}}$$

Le compilateur part typiquement de :

`src/main.rs` ou `src/lib.rs`

et découvre les modules à partir de la structure du crate.

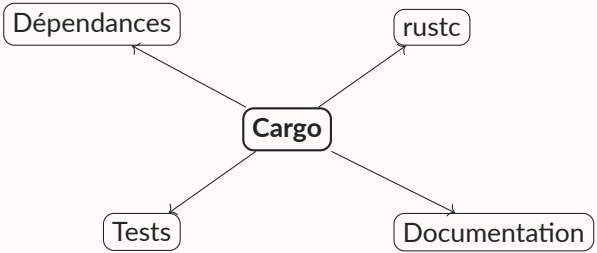
Conséquence

On ne donne normalement pas manuellement tous les fichiers `.rs` au compilateur.

Cargo : le centre du workflow Rust

Cargo combine plusieurs rôles

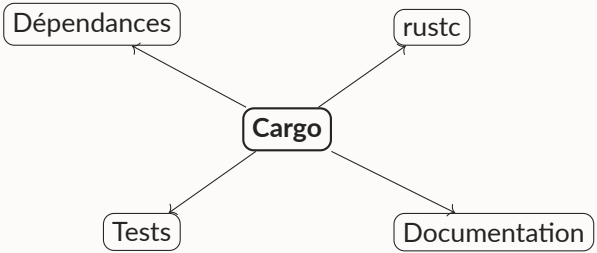
- création de projets
- compilation
- gestion des dépendances
- résolution de versions
- exécution
- tests
- documentation
- intégration avec rustfmt et Clippy



Cargo : le centre du workflow Rust

Cargo combine plusieurs rôles

- création de projets
- compilation
- gestion des dépendances
- résolution de versions
- exécution
- tests
- documentation
- intégration avec rustfmt et Clippy



Idée importante

Pour un développeur Rust, **Cargo est généralement la principale interface avec la toolchain**.

Cargo : décrire le projet

Cargo.toml

Le **manifest** du projet.

```
[package]
name = "my_app"
version = "0.1.0"
edition = "2024"

[dependencies]
serde = "1"
```

Cargo : décrire le projet

Cargo.toml

Le **manifest** du projet.

```
[package]
name = "my_app"
version = "0.1.0"
edition = "2024"

[dependencies]
serde = "1"
```

Cargo.lock

Contient les **versions effectivement résolues** des dépendances. Permet d'obtenir une résolution reproductible du graphe de dépendances.

Cargo : décrire le projet

Cargo.toml

Le **manifest** du projet.

```
[package]
name = "my_app"
version = "0.1.0"
edition = "2024"

[dependencies]
serde = "1"
```

Cargo.lock

Contient les **versions effectivement résolues** des dépendances. Permet d'obtenir une résolution reproductible du graphe de dépendances.

`Cargo.toml`

⇓

résolution des dépendances

⇓

`Cargo.lock`

⇓

build reproductible

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Tester

```
cargo test
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Tester

```
cargo test
```

Qualité

```
cargo fmt
cargo clippy
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Tester

```
cargo test
```

Qualité

```
cargo fmt
cargo clippy
```

Documentation

```
cargo doc
cargo doc --open
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Tester

```
cargo test
```

Qualité

```
cargo fmt
cargo clippy
```

Documentation

```
cargo doc
cargo doc --open
```

Production

```
cargo build --release
```

Cargo : les commandes essentielles

Créer

```
cargo new project
cargo init
```

Développer

```
cargo check
cargo build
cargo run
```

Tester

```
cargo test
```

Qualité

```
cargo fmt
cargo clippy
```

Documentation

```
cargo doc
cargo doc --open
```

Production

```
cargo build --release
```

Réflexe pendant le développement

```
cargo check
```

cargo check ou cargo build ?

cargo check

Analyse le projet

Types
Borrow checker
Traits
Lifetimes

⇒**Rapide**

cargo check ou cargo build ?

cargo check

Analyse le projet

Types
Borrow checker
Traits
Lifetimes

⇒**Rapide**

cargo build

Effectue les vérifications
puis génère le programme

Code machine
Link
Binaire / bibliothèque

⇒**Plus coûteux**

cargo check ou cargo build ?

cargo check

Analyse le projet

Types
Borrow checker
Traits
Lifetimes

⇒**Rapide**

cargo build

Effectue les vérifications
puis génère le programme

Code machine
Link
Binaire / bibliothèque

⇒**Plus coûteux**

Workflow conseillé

- Pendant l'édition : `cargo check`
- Puis lorsque l'on veut exécuter : `cargo run`
- ou produire : `cargo build`

rustfmt : une convention de formatage commune

Rôle

`rustfmt` reformate automatiquement le code Rust selon les conventions de style du langage.

rustfmt : une convention de formatage commune

Rôle

`rustfmt` reformate automatiquement le code Rust selon les conventions de style du langage.

Utilisation

```
cargo fmt
```

Pour vérifier sans modifier les fichiers :

```
cargo fmt --check
```

rustfmt : une convention de formatage commune

Rôle

`rustfmt` reformate automatiquement le code Rust selon les conventions de style du langage.

Utilisation

```
cargo fmt
```

Pour vérifier sans modifier les fichiers :

```
cargo fmt --check
```

Avantages

- style homogène
- moins de discussions de mise en forme
- diffs Git plus propres
- intégration facile dans la CI

rustfmt : une convention de formatage commune

Rôle

`rustfmt` reformate automatiquement le code Rust selon les conventions de style du langage.

Utilisation

```
cargo fmt
```

Pour vérifier sans modifier les fichiers :

```
cargo fmt --check
```

Avantages

- style homogène
- moins de discussions de mise en forme
- diffs Git plus propres
- intégration facile dans la CI

Philosophie

Le développeur ne devrait pas passer du temps à décider :

> “Où mettre ce retour à la ligne ?”

L’outil décide.

Clippy : linter Rust

Question différente de rustc

`rustc` demande principalement : **“Ce programme est-il valide ?”**
`cargo clippy` demande aussi : **“Est-ce une bonne manière d’écrire ce programme ?”**

Clippy : linter Rust

Question différente de rustc

`rustc` demande principalement : **“Ce programme est-il valide ?”**
`cargo clippy` demande aussi : **“Est-ce une bonne manière d’écrire ce programme ?”**

Clippy détecte notamment :

- constructions inutilement complexes
- copies ou allocations superflues
- code non idiomatique
- erreurs logiques fréquentes
- usages suspects de certaines API

Clippy : linter Rust

Question différente de rustc

`rustc` demande principalement : **“Ce programme est-il valide ?”**
`cargo clippy` demande aussi : **“Est-ce une bonne manière d’écrire ce programme ?”**

Clippy détecte notamment :

- constructions inutilement complexes
- copies ou allocations superflues
- code non idiomatique
- erreurs logiques fréquentes
- usages suspects de certaines API

Important

Un warning Clippy n’est pas nécessairement une erreur.
Le développeur doit comprendre le diagnostic avant de modifier le code.

Clippy : niveaux de sévérité

Les diagnostics peuvent être traités avec différents niveaux :

`allow`	ignorer
`warn`	avertissement
`deny`	erreur
`forbid`	impossible à réactiver localement

CI stricte

```
cargo clippy -- -D warnings
```

Tous les warnings deviennent alors des erreurs.

Clippy : niveaux de sévérité

Les diagnostics peuvent être traités avec différents niveaux :

`allow`	ignorer
`warn`	avertissement
`deny`	erreur
`forbid`	impossible à réactiver localement

CI stricte

```
cargo clippy -- -D warnings
```

Tous les warnings deviennent alors des erreurs.

Configuration dans le code

```
#![warn(clippy::pedantic)]
#![allow(clippy::some_lint)]
```

Clippy : niveaux de sévérité

Les diagnostics peuvent être traités avec différents niveaux :

`allow`	ignorer
`warn`	avertissement
`deny`	erreur
`forbid`	impossible à réactiver localement

CI stricte

```
cargo clippy -- -D warnings
```

Tous les warnings deviennent alors des erreurs.

Configuration dans le code

```
#![warn(clippy::pedantic)]
#![allow(clippy::some_lint)]
```

À retenir

Clippy est un **outil d'aide au jugement du programmeur**, pas un remplacement du jugement du programmeur.

rust-analyzer : le moteur de l'IDE

rust-analyzer

Serveur de langage Rust basé sur le protocole **LSP – Language Server Protocol**.

rust-analyzer : le moteur de l'IDE

rust-analyzer

Serveur de langage Rust basé sur le protocole **LSP – Language Server Protocol**.

Navigation

- autocomplétion
- go to definition
- find references
- navigation entre symboles
- documentation intégrée

rust-analyzer : le moteur de l'IDE

rust-analyzer

Serveur de langage Rust basé sur le protocole **LSP – Language Server Protocol**.

Navigation

- autocomplétion
- go to definition
- find references
- navigation entre symboles
- documentation intégrée

Analyse

- types inférés
- erreurs
- inlay hints
- refactorings
- suggestions
- support des macros

rust-analyzer : le moteur de l'IDE

rust-analyzer

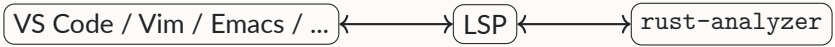
Serveur de langage Rust basé sur le protocole **LSP – Language Server Protocol**.

Navigation

- autocomplétion
- go to definition
- find references
- navigation entre symboles
- documentation intégrée

Analyse

- types inférés
- erreurs
- inlay hints
- refactorings
- suggestions
- support des macros



rust-analyzer vs rustc

rust-analyzer

Optimisé pour :

- interaction immédiate
- édition
- autocomplétion
- navigation
- compréhension du code

IDE

rust-analyzer vs rustc

rust-analyzer

Optimisé pour :

- interaction immédiate
- édition
- autocomplétion
- navigation
- compréhension du code

IDE

rustc

Autorité finale sur :

- validité du programme
- types
- borrow checking
- génération du code
- création du binaire

Compilateur

rust-analyzer vs rustc

rust-analyzer

Optimisé pour :

- interaction immédiate
- édition
- autocomplétion
- navigation
- compréhension du code

IDE

rustc

Autorité finale sur :

- validité du programme
- types
- borrow checking
- génération du code
- création du binaire

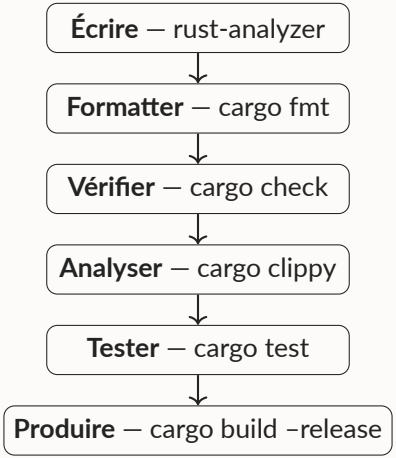
Compilateur

Important

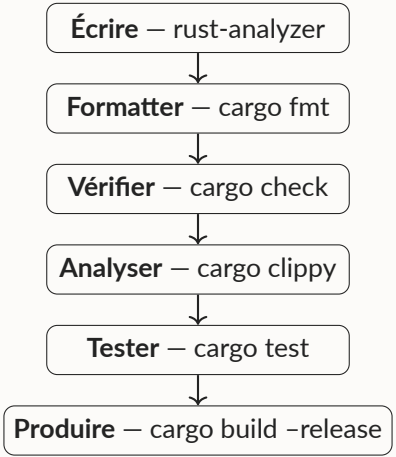
Un éditeur peut afficher une analyse avant que `rustc` ne soit exécuté.
Mais :

rustc reste la référence

Workflow de développement Rust



Workflow de développement Rust



Et au-dessus de tout cela

`rustup` gère la version de Rust, les outils et les plateformes cibles.

La toolchain Rust en une slide

Outil	Question	Rôle
`rustup`	Quel Rust ?	Versions / targets / components
`rustc`	Ce programme est-il valide ?	Compilateur
`cargo`	Comment construire le projet ?	Build / dépendances / tests
`rustfmt`	Comment le présenter ?	Formatage
`clippy`	Est-il idiomatique ?	Analyse statique / lints
`rust-analyzer`	Comment m'aider à l'écrire ?	IDE / LSP

La toolchain Rust en une slide

Outil	Question	Rôle
`rustup`	Quel Rust ?	Versions / targets / components
`rustc`	Ce programme est-il valide ?	Compilateur
`cargo`	Comment construire le projet ?	Build / dépendances / tests
`rustfmt`	Comment le présenter ?	Formatage
`clippy`	Est-il idiomatique ?	Analyse statique / lints
`rust-analyzer`	Comment m'aider à l'écrire ?	IDE / LSP

À retenir

La qualité de l'expérience développeur fait partie du design de l'écosystème Rust.

Ces outils sont conçus pour être utilisés **ensemble**, pas indépendamment.

Rust : Les bases

Mutabilité — le code fautif et le diagnostic

```rust
fn main() {
    let x = 42;
    x = 43;
    println!("{x}");
}
```

rustc 1.98.1 — diagnostic principal

```
error[E0384]: cannot assign twice to
     immutable variable `x`

help: consider making this binding mutable:
    let mut x = 42;
```

À retenir

En Rust, l'immuabilité est le défaut : une mutation doit être visible dans le contrat local.

Mutabilité — correction et compilation propre

```rust
fn main() {
    let mut x = 42;
    x = 43;
    println!("{x}");
}
```

```
$ rustc --edition 2024 mutability.rs
# aucune erreur : exit status 0

$ ./mutability
43
```

À retenir

`mut` ne change pas le type de `x` ; il change le droit de modifier ce binding.

Shadowing : même nom, nouveau binding

```
let x = 42;
let x = x + 1;
let x = x.to_string();

println!("{x}");
```

Ce n'est pas...

une mutation du même emplacement.

C'est...

un nouveau binding, qui peut même avoir un autre type.

Expressions — un point-virgule change la valeur du bloc

```rust
fn square(x: i32) -> i32 {
    x * x;
}
```

```
error[E0308]: mismatched types
  expected `i32`
     found `()`

help: remove this semicolon to return this
      value
```

À retenir

`x * x` est une expression de type `i32` ; `x * x;` devient une instruction et le bloc vaut `()`.

Expressions — correction

```rust
fn square(x: i32) -> i32 {
    x * x
}

fn main() {
    println!("{}", square(7));
}
```

```
$ rustc --edition 2024 expressions.rs
# aucune erreur : exit status 0

$ ./expressions
49
```

Structs et méthodes

```
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    fn norm(&self) -> f64 {
        (self.x * self.x + self.y * self.y).sqrt()
    }
}
```

Question à garder en tête

Que signifie exactement `&self` ? Nous le verrons plus tard.

Objet — une méthode `&self` ne peut pas muter l'état

```rust
impl Point {
    fn move_x(&self, dx: f64) {
        self.x += dx;
    }
}
```

```
error[E0594]: cannot assign to `self.x`,
     which is behind a `&` reference

help: consider changing this to be a
     mutable reference:
    fn move_x(&mut self, dx: f64)
```

À retenir

`&self` promet une lecture partagée ; `&mut self` demande un accès exclusif temporaire.

Objet — correction : `&mut self` et binding mutable

```rust
impl Point {
    fn move_x(&mut self, dx: f64) {
        self.x += dx;
    }
}

fn main() {
    let mut p = Point { x: 3.0, y: 4.0 };
    p.move_x(1.0);
}
```

```
$ rustc --edition 2024 object_mut.rs
# aucune erreur : exit status 0
```

Objet en Rust : pas de classes, mais de l'encapsulation

Rust n'a pas

- de mot-clé `class` ;
- d'héritage de classes ;
- de constructeurs/destructeurs au sens Java/C++ ;
- de hiérarchie racine de type `Object`.

Rust fournit

- données : `struct` / `enum` ;
- méthodes : blocs `impl` ;
- encapsulation : visibilité des modules ;
- polymorphisme : `trait` ;
- composition explicite.

À retenir

Rust permet une programmation orientée objet, mais **pas un modèle objet fondé sur l'héritage de classes**.

Construire un « objet » : état + invariants + méthodes

```rust
pub struct BankAccount {
    balance: i64, // privé au module
}

impl BankAccount {
    pub fn new() -> Self {
        Self { balance: 0 }
    }

    pub fn deposit(&mut self, amount: i64) {
        assert!(amount > 0);
        self.balance += amount;
    }

    pub fn balance(&self) -> i64 {
        self.balance
    }
}

fn main() {
    let mut x = BankAccount::new();
    BankAccount::deposit(&mut x, 42);
    println!("{}", BankAccount::balance(&x));
}
```

À retenir

Une fonction associée nommée `new` est une **convention**, pas un constructeur spécial du langage.

Trois formes de méthodes : propriété, lecture, mutation

```rust
impl Document {
    fn id(&self) -> u64 { self.id }          // emprunte

    fn rename(&mut self, name: String) {     // emprunt mutable
        self.name = name;
    }

    fn into_bytes(self) -> Vec<u8> {         // consomme l'objet
        self.data
    }
}
```

`&self`	`&mut self`	`self`
lecture partagée	mutation exclusive	transfert d'ownership

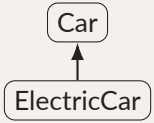
À retenir

La signature d'une méthode décrit aussi son **effet sur la propriété de l'objet**.

Pas d'héritage de classes : préférer la composition

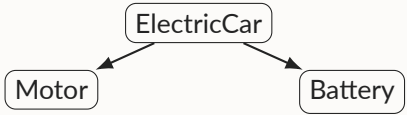
Réflexe Java/C++

```
class ElectricCar extends Car
```



Réflexe Rust

```
struct ElectricCar { motor: Motor,
        battery: Battery }
```



À retenir

Réutiliser le **comportement** avec des traits ; réutiliser l'**état** par composition.

Pourquoi Rust n'a pas d'héritage de classes ?

Choix de conception

Rust ne propose pas d'héritage de classes traditionnel mais privilégie :

- la **composition**
- les **traits**
- les **enums**
- le polymorphisme explicite

Pourquoi Rust n'a pas d'héritage de classes ?

Choix de conception

Rust ne propose pas d'héritage de classes traditionnel mais privilégie :

- la **composition**
- les **traits**
- les **enums**
- le polymorphisme explicite

Problèmes fréquents de l'héritage

- hiérarchies profondes et rigides
- couplage fort entre classes
- comportement hérité implicitement
- difficulté de connaître les méthodes appelées
- problème du "fragile base class"
- héritage multiple et ambiguïtés

Pourquoi Rust n'a pas d'héritage de classes ?

Choix de conception

Rust ne propose pas d'héritage de classes traditionnel mais privilégie :

- la **composition**
- les **traits**
- les **enums**
- le polymorphisme explicite

Problèmes fréquents de l'héritage

- hiérarchies profondes et rigides
- couplage fort entre classes
- comportement hérité implicitement
- difficulté de connaître les méthodes appelées
- problème du "fragile base class"
- héritage multiple et ambiguïtés

Approche Rust

- **Composition** : construire avec des composants
- **Traits** : partager des comportements
- **Enums** : modéliser un ensemble fermé de variantes
- **dyn Trait** : polymorphisme dynamique si nécessaire

Pourquoi Rust n'a pas d'héritage de classes ?

Choix de conception

Rust ne propose pas d'héritage de classes traditionnel mais privilégie :

- la **composition**
- les **traits**
- les **enums**
- le polymorphisme explicite

Problèmes fréquents de l'héritage

- hiérarchies profondes et rigides
- couplage fort entre classes
- comportement hérité implicitement
- difficulté de connaître les méthodes appelées
- problème du "fragile base class"
- héritage multiple et ambiguïtés

Approche Rust

- **Composition** : construire avec des composants
- **Traits** : partager des comportements
- **Enums** : modéliser un ensemble fermé de variantes
- **dyn Trait** : polymorphisme dynamique si nécessaire

Avantage principal

Les relations entre types sont plus explicites et moins couplées.

Pourquoi Rust n'a pas d'héritage de classes ?

Choix de conception

Rust ne propose pas d'héritage de classes traditionnel mais privilégie :

- la **composition**
- les **traits**
- les **enums**
- le polymorphisme explicite

Problèmes fréquents de l'héritage

- hiérarchies profondes et rigides
- couplage fort entre classes
- comportement hérité implicitement
- difficulté de connaître les méthodes appelées
- problème du "fragile base class"
- héritage multiple et ambiguïtés

Approche Rust

- **Composition** : construire avec des composants
- **Traits** : partager des comportements
- **Enums** : modéliser un ensemble fermé de variantes
- **dyn Trait** : polymorphisme dynamique si nécessaire

Avantage principal

Les relations entre types sont plus explicites et moins couplées.

Idée clé

Favoriser la composition plutôt que l'héritage : On réutilise des **capacités** plutôt qu'une hiérarchie de classes.

Polymorphisme objet : fermé avec `enum`, ouvert avec `trait`

Univers fermé : toutes les variantes connues

```
enum Shape {
    Circle(Circle),
    Rectangle(Rectangle),
}

match shape {
    Shape::Circle(c) => ...,
    Shape::Rectangle(r) => ...,
}
```

Univers ouvert : de nouveaux types peuvent implémenter

```
trait Draw {
    fn draw(&self);
}

fn render(x: &dyn Draw) {
    x.draw();
}
```

À retenir

`enum` donne l'exhaustivité ; `trait` donne l'extensibilité. Nous approfondirons les traits plus tard.

Les enums Rust portent des données

```
enum Shape {
    Circle(f64),
    Rectangle { width: f64, height: f64 },
    Point,
}
```

Réflexe C

`tag + union`

Réflexe F#

discriminated union / type somme

Le **tag** et les données associées font partie du même type.

`match` — oublier un cas est une erreur de compilation

```
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle(r) =>
            std::f64::consts::PI * r *
    r,
        Shape::Rectangle { width,
    height } =>
            width * height,
        // Shape::Point oublié
    }
}
```

```
error[E0004]: non-exhaustive patterns: `&
    Shape::Point` not covered

help: ensure that all possible cases are
    being handled
```

`match` — correction exhaustive

```
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle(r) =>
            std::f64::consts::PI * r * r,
        Shape::Rectangle { width, height }
    =>
            width * height,
        Shape::Point => 0.0,
    }
}
```

```
$ rustc --edition 2024 match.rs
# aucune erreur : exit status 0
```

À retenir

Ajouter une variante d'`enum` force les `match` exhaustifs à être réexaminés.

`String` vs `&str`

```
let a = "hello";
let mut b = String::from("hello");

b.push_str(" Rust");
```

Valeur	Type
a	&str
b	String

Intuition provisoire

`String` : chaîne **possédée**.
`&str` : **vue** sur une chaîne.

UTF-8 — une `String` ne s'indexe pas par entier

```rust
fn main() {
    let s = String::from("été");
    println!("{}", s[0]);
}
```

```
error[E0277]: the type `str` cannot be
     indexed by `{integer}`

help: string indices are ranges of `usize`
```

À retenir

Un index entier serait ambigu : octet, scalar Unicode ou grapheme utilisateur ? Rust oblige à choisir l'abstraction.

UTF-8 — correction : choisir `chars()` ou les octets

```rust
fn main() {
    let s = String::from("été");
    println!("{}", s.chars().next().
    unwrap());
}
```

```
$ rustc --edition 2024 utf8.rs
# aucune erreur : exit status 0

$ ./utf8
é
```

Collections : juste assez pour avancer

```rust
let values = vec![10, 20, 30];
for value in &values {
    println!("{value}");
}

let mut ages = std::collections::HashMap::new();
ages.insert("Alice", 23);
ages.insert("Bob", 24);
```

Pourquoi le compilateur refuse-t-il ?

Quel est le type de `ages.get("Alice")` ?

Option<&i32>

Nous y reviendrons plus tard.

Move — utiliser l'ancien propriétaire est interdit

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;

    println!("{s2}");
    println!("{s1}");
}
```

```
error[E0382]: borrow of moved value: `s1`

  let s2 = s1;
           -- value moved here
```

À retenir

Une `String` n'est pas `Copy` : l'affectation transfère son ownership.

Move — deux corrections, deux sémantiques

Un seul propriétaire : emprunter

```
let s1 = String::from("hello");
let s2 = &s1;
println!("{s1} {s2}");
```

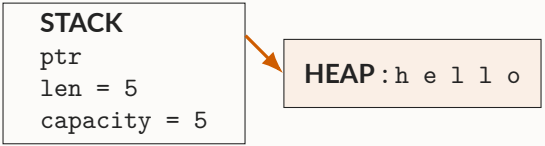
Deux valeurs possédées : cloner

```
let s1 = String::from("hello");
let s2 = s1.clone();
println!("{s1} {s2}");
```

```
$ rustc --edition 2024 move_fixed.rs
# aucune erreur : exit status 0
```

Rust : La possession

Stack, heap, et responsabilité de destruction



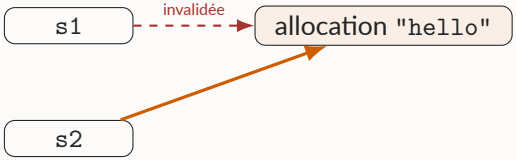
C	`malloc/free`
Java	GC
C++	RAII
Rust	ownership + `Drop`

À retenir

Le compilateur doit toujours pouvoir répondre : **qui libère cette ressource ?**

Move semantics : un propriétaire à la fois

```
let s1 = String::from("hello");
let s2 = s1;
```



À retenir

Pas de copie profonde implicite. Pas de double destruction possible.

Passage par valeur — la fonction consomme la `String`

```rust
fn consume(s: String) {
    println!("{s}");
}

fn main() {
    let name = String::from("Ferris");
    consume(name);
    println!("{name}");
}
```

```
error[E0382]: borrow of moved value: `name`

  consume(name);
          ---- value moved here
```

Passage par référence — correction idiomatique

```rust
fn observe(s: &str) {
    println!("{s}");
}

fn main() {
    let name = String::from("Ferris");
    observe(&name);
    println!("{name}");
}
```

```
$ rustc --edition 2024 borrow_arg.rs
# aucune erreur : exit status 0
```

À retenir

La signature dit l'intention : `String` = prendre ; `&str` = observer.

`Copy` et `Clone` : deux intentions différentes

```rust
let a = 42;
let b = a;

println!("{a} {b}");
```

Copy

```rust
let s1 = String::from("hello");
let s2 = s1.clone();

println!("{s1} {s2}");
```

Clone explicite

Attention

`clone()` est utile, mais ce n'est pas un bouton « tais-toi borrow checker ».

Borrowing : prêter sans transférer

```rust
fn length(s: String) -> usize {
    s.len()
}
```

consomme

```rust
fn length(s: &str) -> usize {
    s.len()
}
```

emprunte

```rust
let name = String::from("Ferris");
let n = length(&name);
println!("{name} : {n}");
```

À retenir

`&T` permet d'observer sans devenir propriétaire.

La règle à retenir : aliasing XOR mutability

plusieurs `&T`
lecteurs

OU

un `&mut T`
écrivain exclusif

Aliasing XOR Mutability

Borrowing — deux `&mut` actifs sont interdits

```rust
let mut s = String::from("hello");

let a = &mut s;
let b = &mut s;

println!("{a} {b}");
```

```
error[E0499]: cannot borrow `s` as mutable
     more than once at a time

first mutable borrow occurs here
second mutable borrow occurs here
```

Borrowing — correction par scopes d'emprunt disjoints

```rust
let mut s = String::from("hello");

{
    let a = &mut s;
    a.push('!');
}

let b = &mut s;
b.push('?');
```

```
$ rustc --edition 2024 two_mut.rs
# aucune erreur : exit status 0
```

À retenir

L'exclusivité est temporaire : le second emprunt devient possible dès que le premier n'est plus actif.

Borrow checker — une référence peut être invalidée par `push`

```rust
let mut values = vec![10, 20, 30];

let first = &values[0];
values.push(40);

println!("{first}");
```

```
error[E0502]: cannot borrow `values` as
     mutable because it is also borrowed as
     immutable

immutable borrow occurs here
mutable borrow occurs here
immutable borrow later used here
```

Borrow checker — correction : terminer l'emprunt avant la mutation

```rust
let mut values = vec![10, 20, 30];

let first = values[0]; // i32 est Copy
values.push(40);

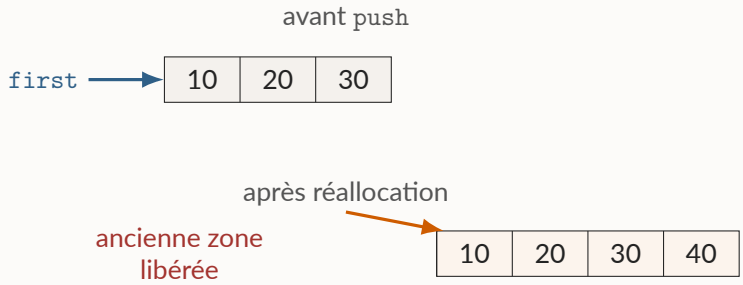
println!("{first}");
```

```
$ rustc --edition 2024 vec_borrow.rs
# aucune erreur : exit status 0
```

À retenir

Ici on copie la petite valeur, pas le `Vec`. Autre solution : utiliser la référence avant le `push`.

Ce que Rust empêche ici



À retenir

En C/C++, une référence/pointeur pourrait devenir ”dangling”. Rust refuse la situation avant exécution.

Non-lexical lifetimes : l'emprunt cesse quand il n'est plus utile

```
let mut s = String::from("hello");

let r = &s;
println!("{r}");

s.push('!');
```

Pourquoi le compilateur ne hurle pas ?

`r` est encore dans le bloc. Pourquoi la mutation peut-elle être acceptée ?

À retenir

Rust raisonne sur l'usage effectif de la référence, pas seulement sur les accolades.

Slices : emprunter une partie

```rust
let values = [10, 20, 30, 40];
let middle = &values[1..3]; // &[i32]

let message = String::from("hello world");
let hello = &message[..5]; // &str
```

API trop spécifique

```rust
fn sum(v: &Vec<i32>)
```

API plus générale

```rust
fn sum(v: &[i32])
```

Idée clé

Une fonction devrait demander **la vue la plus générale dont elle a besoin**, pas le conteneur concret.

Lifetime — impossible de retourner une référence vers une locale

```rust
fn broken() -> &String {
    let s = String::from("hello");
    &s
}
```

```
error[E0106]: missing lifetime specifier

error[E0515]: cannot return reference to
     local variable `s`
```

À retenir

Aucune annotation de lifetime ne peut rendre valide une référence vers une valeur détruite à la sortie de la fonction.

Lifetime — correction : retourner la valeur possédée

```rust
fn make_string() -> String {
    String::from("hello")
}

fn main() {
    let s = make_string();
    println!("{s}");
}
```

```
$ rustc --edition 2024 lifetime_owned.rs
# aucune erreur : exit status 0
```

Lifetime — le résultat emprunte lequel des deux paramètres ?

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() >= y.len() { x } else { y
     }
}
```

```
error[E0106]: missing lifetime specifier

help: consider introducing a named
     lifetime parameter
```

Lifetime — correction : exprimer la relation

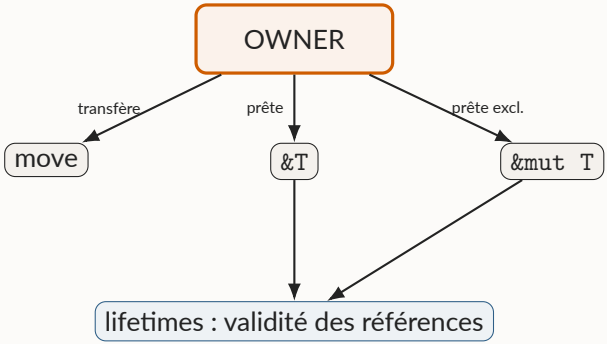
```rust
fn longest<'a>(x: &'a str, y: &'a str) ->
    &'a str {
    if x.len() >= y.len() { x } else { y }
}
```

```
$ rustc --edition 2024 longest.rs
# aucune erreur : exit status 0
```

À retenir

`'a` ne prolonge rien : il décrit la relation nécessaire entre entrées et sortie.

Résumé



plusieurs lecteurs **OU** un écrivain

Rust : Erreurs, Traits et Tests

`Option<T>` : l'absence fait partie du type

```rust
fn find_even(values: &[i32]) -> Option<i32> {
    for &x in values {
        if x % 2 == 0 {
            return Some(x);
        }
    }
    None
}
```

Deux états explicites

`Some(T)`
`None`

Effet pédagogique

L'appelant ne peut pas oublier que l'absence est possible.

Pattern matching ergonomique

```rust
match value {
    Some(x) => println!("{x}"),
    None => println!("nothing"),
}
```

```rust
if let Some(x) = value {
    println!("{x}");
}
```

```rust
let Some(x) = value else {
    return;
};
```

```rust
let x = value.unwrap_or(0);
```

`unwrap()` ?

Pas interdit. Il signifie : « ce cas est impossible, ou je consens à paniquer ».

`Result<T,E>` : erreurs comme valeurs

```rust
use std::fs;

fn read_config(path: &str)
    -> Result<String, std::io::Error>
{
    fs::read_to_string(path)
}
```

Java	exceptions
C	codes d'erreur / conventions
F#	`Result<'T,'E>`
Rust	`Result<T,E>` dans la signature

L'opérateur ? : propager sans bruit

```rust
fn load(path: &str)
 -> Result<String, std::io::Error>
{
    match std::fs::read_to_string(
    path) {
        Ok(s) => Ok(s),
        Err(e) => Err(e),
    }
}
```

```rust
fn load(path: &str)
 -> Result<String, std::io::Error>
{
    let s = std::fs::read_to_string(
    path)?;
    Ok(s)
}
```

À retenir

? ne cache pas l'erreur : il automatise sa propagation.

? — une fonction doit pouvoir propager l'erreur

```rust
fn main() {
    let contents =
        std::fs::read_to_string("config.
    txt")?;
    println!("{contents}");
}
```

```
error[E0277]: the `?` operator can only
    be used in a function that returns `
    Result` or `Option` (or another type
    that implements `FromResidual`)
```

? — correction : rendre le contrat visible dans le type

```rust
fn main() -> Result<(), Box<dyn std::error::
    Error>> {
    let contents =
        std::fs::read_to_string("config.txt")
    ?;
    println!("{contents}");
    Ok(())
}
```

```
$ rustc --edition 2024 question.rs
# aucune erreur : exit status 0
```

Traits : décrire un comportement

```rust
trait Area {
    fn area(&self) -> f64;
}

struct Circle { radius: f64 }

impl Area for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}
```

À retenir

Analogie utile : interface Java / concept C++ / contraintes génériques F#. Aucune n'est exacte.

Trait bound — un appel incompatible est refusé

```rust
fn print_area<T: Area>(shape: &T) {
    println!("{}", shape.area());
}

fn main() {
    print_area(&42);
}
```

```
error[E0277]: the trait bound `{integer}:
      Area` is not satisfied

required by a bound in `print_area`
```

Trait bound — correction : passer un type qui implémente le contrat

```
let c = Circle { radius: 2.0 };
print_area(&c);
```

```
$ rustc --edition 2024 trait_bound.rs
# aucune erreur : exit status 0
```

À retenir

Le trait bound documente le contrat et le rend vérifiable à la compilation.

Deux formes de polymorphisme

Statique

`T: Trait` Type concret connu à la compilation. Monomorphisation fréquente.

Dynamique

`dyn Trait` Trait object ; dispatch dynamique via une indirection.

Rust laisse le choix explicite selon le besoin.

Trait objects : collections hétérogènes

```
trait Describe {
    fn describe(&self) -> String;
}

let objects: Vec<Box<dyn Describe>> = vec![
    Box::new(circle),
    Box::new(rectangle),
];
```

À ne pas faire aujourd'hui

Entrer dans tous les détails de vtable, object safety, coercions et lifetimes de trait objects.

Closures et iterators : même idée, autre coût mental

```rust
let mut result = Vec::new();
for x in &values {
    if x % 2 == 0 {
        result.push(x * x);
    }
}
```

```rust
let result: Vec<i32> = values
    .iter()
    .filter(|x| **x % 2 == 0)
    .map(|x| x * x)
    .collect();
```

À retenir

Pipelines familiers de F#, Java, JS... mais toujours soumis à l'ownership.

`into_iter()` — consommer la collection est visible

```rust
let values = vec![1, 2, 3];
let total: i32 = values.into_iter().sum
    ();

println!("{total}");
println!("{values:?}");
```

```
error[E0382]: borrow of moved value: `
    values`

`into_iter` takes ownership of the
    receiver `self`, which moves `values`
```

Iterator — correction : emprunter avec `iter()`

```rust
let values = vec![1, 2, 3];
let total: i32 = values.iter().copied().
    sum();

println!("{total}");
println!("{values:?}");
```

```
$ rustc --edition 2024 iterator.rs
# aucune erreur : exit status 0
```

À retenir

`iter()` emprunte ; `into_iter()` consomme.

Structurer un vrai projet

```
my_project/
   Cargo.toml
   src/
      main.rs
      lib.rs
      parser.rs
   tests/
       integration.rs
```

- package ;
- crate ;
- module ;
- `pub` ;
- `use`.

Objectif CM

Savoir lire la structure ; les subtilités du système de modules viendront en TP.

Tests : intégrés à la boucle de développement

```rust
fn add(a: i32, b: i32) -> i32 { a + b }

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn addition_works() {
        assert_eq!(add(2, 3), 5);
    }
}
```

`cargo fmt` `cargo clippy` `cargo test`

Tests : intégrés à la boucle de développement

```rust
fn add(a: i32, b: i32) -> i32 {a + b}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn addition_works() {
        assert_eq!(add(2, 3), 5);
    }
}
```

`cargo fmt` `cargo clippy` `cargo test`

À retenir

Formatage, analyse statique et tests font partie du workflow standard Rust.

Rust : la concurrence

Pourquoi autant de « pointeurs intelligents » ?

Besoin	`Type`
ownership d'une allocation heap	`Box<T>`
ownership partagé mono-thread	`Rc<T>`
ownership partagé multi-thread	`Arc<T>`
mutation intérieure vérifiée à l'exécution	`RefCell<T>`
accès mutable synchronisé entre threads	`Mutex<T>`

À retenir

Ne mémorisez pas cinq pointeurs : mémorisez cinq **problèmes différents**.

`Box<T>` : casser la récursion de taille

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}
```

Pourquoi le compilateur refuse-t-il ?

Pourquoi un type récursif a-t-il besoin d'une indirection ?



À retenir

Un type récursif ne peut pas contenir directement une copie de lui-même. `Box<T>` ajoute une indirection de taille fixe et stocke la valeur sur le heap.

`Rc<T>` : partager l'ownership dans un thread

```
use std::rc::Rc;

let a = Rc::new(String::from("shared"));
let b = Rc::clone(&a);
let c = Rc::clone(&a);

println!("{}", Rc::strong_count(&a));
```

À retenir

`Rc::clone` incrémente un compteur ; ce n'est pas une copie profonde de la valeur.

`RefCell<T>` — le code compile, mais peut paniquer

```
use std::cell::RefCell;

let values = RefCell::new(vec![1, 2,
    3]);
let first = values.borrow_mut();
let second = values.borrow_mut();
println!("{first:?} {second:?}");
```

```
$ rustc --edition 2024 refcell.rs
# compilation réussie
```

```
$ ./refcell
thread 'main' panicked ...
already borrowed: BorrowMutError
```

À retenir

`RefCell` conserve les règles de borrowing mais les vérifie à l'exécution.

`RefCell<T>` — correction : ne pas chevaucher les emprunts mutables

```rust
let values = RefCell::new(vec![1, 2, 3]);

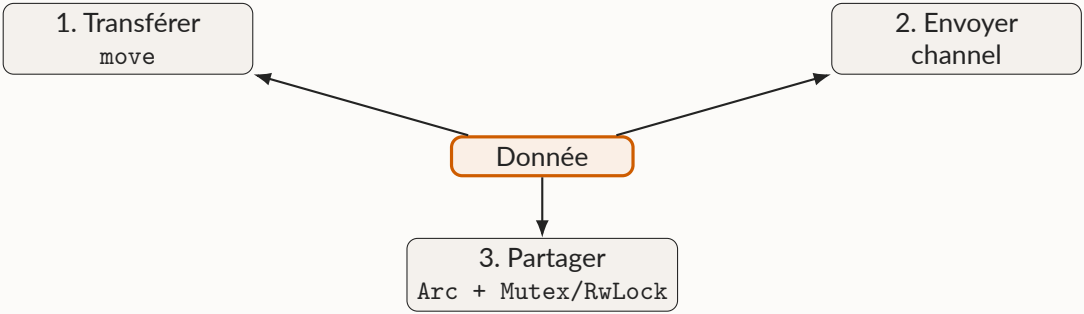
{
    let mut first = values.borrow_mut();
    first.push(4);
}

let second = values.borrow();
println!("{second:?}");
```

```
$ rustc --edition 2024 refcell_fixed.rs
# aucune erreur : exit status 0

$ ./refcell_fixed
[1, 2, 3, 4]
```

Threading en Rust : trois manières de faire circuler les données



À retenir

Avant de choisir une primitive de synchronisation, choisir **le modèle de propriété des données entre threads**.

`std::thread` : de vrais threads système

Le modèle standard

- `thread::spawn` crée un thread natif ;
- le système d'exploitation ordonnance les threads ;
- plusieurs threads peuvent réellement s'exécuter en parallèle sur plusieurs cœurs ;
- `JoinHandle` permet d'attendre la terminaison et de récupérer un résultat.

Rust ajoute des contraintes statiques

La closure envoyée au thread doit respecter les règles de durée de vie et les propriétés `Send`/`Sync`. Beaucoup de data races deviennent donc des erreurs de compilation.

Ce que Rust ne garantit pas

Pas de garantie contre les deadlocks, starvation, livelocks ou erreurs de logique concurrente.

Thread — une closure empruntée pourrait survivre à la fonction

```rust
use std::thread;

let values = vec![1, 2, 3];
let handle = thread::spawn(|| {
    println!("{values:?}");
});
handle.join().unwrap();
```

```
error[E0373]: closure may outlive the
     current function, but it borrows `
     values`, which is owned by the current
      function

help: use the `move` keyword
```

Thread — correction : transférer les captures avec `move`

```rust
use std::thread;

let values = vec![1, 2, 3];
let handle = thread::spawn(move || {
    println!("{values:?}");
});
handle.join().unwrap();
```

```
$ rustc --edition 2024 thread_move.rs
# aucune erreur : exit status 0

$ ./thread_move
[1, 2, 3]
```

Après `move`, le parent n'est plus propriétaire

```
let values = vec![1, 2, 3];
let handle = std::thread::spawn(move ||
     {
    println!("{values:?}");
});

println!("{values:?}");
handle.join().unwrap();
```

```
error[E0382]: borrow of moved value: `
     values`

value moved into closure here
value borrowed here after move
```

Après `move` — correction : décider qui doit posséder la donnée

```rust
let values = vec![1, 2, 3];
let values_for_thread = values.clone();

let handle = std::thread::spawn(move || {
    println!("{values_for_thread:?}");
});
println!("{values:?}");
handle.join().unwrap();
```

```
$ rustc --edition 2024 thread_clone.rs
# aucune erreur : exit status 0
```

À retenir

Le clone est ici une décision explicite : nous voulons réellement deux vecteurs indépendants.

`thread::scope` : emprunter sans imposer `'static`

```
let values = vec![10, 20, 30, 40];

std::thread::scope(|scope| {
    let left = &values[..2];
    let right = &values[2..];

    let a = scope.spawn(|| left.iter().sum::<i32>());
    let b = scope.spawn(|| right.iter().sum::<i32>());

    println!("{}", a.join().unwrap() + b.join().unwrap());
});
```

À retenir

`thread::scope` garantit que les threads enfants sont terminés avant la fin du bloc. Ils peuvent donc emprunter des variables locales du thread parent en toute sécurité, sans Arc et sans transférer leur ownership.

Channel — envoyer une valeur transfère son ownership

```rust
let (tx, rx) = std::sync::mpsc::channel()
    ;

std::thread::spawn(move || {
    let message = String::from("hello");
    tx.send(message).unwrap();
    println!("{message}");
});

println!("{}", rx.recv().unwrap());
```

```
error[E0382]: borrow of moved value: `
     message`

value moved here:
    tx.send(message)
```

Channel — correction : l'émetteur abandonne la valeur

```rust
let (tx, rx) = std::sync::mpsc::channel();

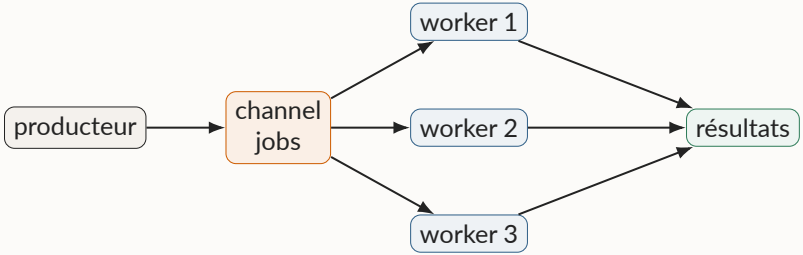
let h = std::thread::spawn(move || {
    let message = String::from("hello");
    tx.send(message).unwrap();
});

println!("{}", rx.recv().unwrap());
h.join().unwrap();
```

```
$ rustc --edition 2024 channel.rs
# aucune erreur : exit status 0

$ ./channel
hello
```

Channels : une architecture de workers sans mémoire mutable partagée



Avantage

Le transfert d'ownership réduit la quantité d'état partagé à synchroniser.

À savoir

`std::sync::mpsc` est simple ; l'écosystème propose aussi d'autres channels selon les besoins de performance et de topologie.

`Rc<T>` — le type dit qu'il n'est pas thread-safe

```rust
use std::rc::Rc;

let value = Rc::new(42);
let value2 = Rc::clone(&value);

std::thread::spawn(move || {
    println!("{value2}");
});
```

```
error[E0277]: `Rc<i32>` cannot be sent
     between threads safely

within this closure, the trait `Send` is
     not implemented for `Rc<i32>`
```

`Rc<T>` — correction : `Arc<T>` pour le partage inter-thread

```rust
use std::sync::Arc;

let value = Arc::new(42);
let value2 = Arc::clone(&value);

let h = std::thread::spawn(move || {
    println!("{value2}");
});
h.join().unwrap();
```

```
$ rustc --edition 2024 arc_thread.rs
# aucune erreur : exit status 0
```

À retenir

`Arc` paie un compteur de références atomique pour permettre le partage entre threads.

`Arc<Mutex<T>>` : deux besoins, deux abstractions

```rust
use std::sync::{Arc, Mutex};

let counter = Arc::new(Mutex::new(0));
let mut handles = Vec::new();

for _ in 0..10 {
    let counter = Arc::clone(&counter);
    handles.push(std::thread::spawn(move || {
        let mut n = counter.lock().unwrap();
        *n += 1;
    }));
}
```

Arc
ownership partagé

Mutex
accès mutable exclusif

Un `Mutex` évite les data races, pas les deadlocks

```
let a = Arc::new(Mutex::new(...));
let b = Arc::new(Mutex::new(...));

// Thread 1
let _a = a.lock().unwrap();
let _b = b.lock().unwrap();

// Thread 2
let _b = b.lock().unwrap();
let _a = a.lock().unwrap();
```

Possible

Deux threads peuvent attendre indéfiniment l'un sur l'autre.

Bonnes pratiques

- ordre global des verrous ;
- scopes de verrou courts ;
- éviter le partage quand un channel suffit ;
- documenter les invariants de synchronisation.

`Send` et `Sync` : des propriétés dans le système de types

Send

Une valeur peut être transférée de manière sûre vers un autre thread.

Sync

Une référence partagée vers la valeur peut être utilisée de manière sûre depuis plusieurs threads.

`Rc<T>` : mauvais outil pour le multi-thread.

`Arc<T>` : compteur de références atomique.

Au-delà de `Mutex` : choisir la bonne primitive

`Primitive`	Cas typique
`Mutex<T>`	une donnée mutable, accès exclusif relativement grossier
`RwLock<T>`	beaucoup de lecteurs, peu d'écrivains
`Atomic*`	compteurs / flags / états simples, opérations lock-free ciblées
`Barrier`	synchroniser plusieurs threads sur une phase
`Condvar`	attendre qu'une condition protégée par mutex devienne vraie
`channel`	transmettre des données plutôt que partager leur mutation

À retenir

Les atomics ne sont pas « un Mutex plus rapide » : leur correction dépend du **modèle mémoire** et du choix d'ordering.

Parallélisme de données : l'écosystème peut simplifier le threading

Séquentiel

```
let sum: u64 = values
    .iter()
    .map(|x| expensive(x))
    .sum();
```

Avec Rayon

```
use rayon::prelude::*;

let sum: u64 = values
    .par_iter()
    .map(|x| expensive(x))
    .sum();
```

À retenir

Pour un problème CPU data-parallel, une abstraction de haut niveau peut être préférable à la gestion manuelle d'un pool de threads.

Threads, Rayon ou async ?

Besoin	Outil naturel	Intuition
quelques tâches CPU indépendantes	`std::thread`	threads OS explicites
beaucoup de calcul sur collections	Rayon	parallélisme de données / pool
beaucoup d'I/O concurrentes	async + runtime	tâches légères, attente coopérative
état partagé	`Arc` + verrou / atomics	synchronisation explicite
transfert de jobs/résultats	channels	message passing

À retenir

« Concurrence » n'est pas synonyme de « async ». Rust offre plusieurs modèles, tous connectés à l'ownership.

Async : ne pas confondre thread et tâche

```rust
async fn fetch_data() -> Result<String, Error> {
    // I/O asynchrone...
}

let data = fetch_data().await?;
```

`async fn`

Produit un calcul représenté par un `Future`.

Runtime

En pratique : Tokio est un choix courant pour exécuter les tâches asynchrones.

Async : ce que l'on garde pour le TP



Pas aujourd'hui

`Pin`, `Poll`, wakers, machine à états générée, détails de l'executor.

`unsafe` : une frontière de preuve

```rust
fn get(values: &[i32], index: usize) -> &i32 {
    assert!(index < values.len());

    unsafe {
        values.get_unchecked(index)
    }
}
```

À retenir

`unsafe` n'éteint pas Rust. Il transfère au programmeur la preuve d'un petit ensemble d'invariants.

API sûre → petite frontière unsafe → opération bas niveau

FFI C : Rust 2024 rend la frontière explicite

```rust
unsafe extern "C" {
    fn external_function(x: i32) -> i32;
}

#[unsafe(no_mangle)]
pub extern "C" fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

Message

Au bord du langage, le compilateur ne peut plus vérifier tous les contrats : la frontière doit être visible et documentée.

Synthèse : diagnostiquer un programme invalide

```rust
use std::thread;

fn main() {
    let mut names = vec![
        String::from("Alice"),
        String::from("Bob"),
    ];

    let first = &names[0];
    names.push(String::from("Charlie"));

    let handle = thread::spawn(|| {
        println!("{names:?}");
    });

    println!("first = {first}");
    handle.join().unwrap();
}
```

Pourquoi le compilateur refuse-t-il ?

Où sont les conflits d'emprunt, de mutation et de durée de vie ?

Les 12 réflexes à emporter en TP

1. Qui possède la valeur ?
2. `T` peut transférer ; `&T` prête ; `&mut T` prête exclusivement.
3. Plusieurs lecteurs **ou** un écrivain.
4. Préférer `&str` à `&String`, `&[T]` à `&Vec<T>` pour observer.
5. Une lifetime décrit une **relation**, elle ne rallonge rien.
6. `Option` = absence explicite ; `Result` = erreur explicite.
7. `?` propage ; `unwrap()` revendique un invariant ou accepte une panique.
8. `iter()` emprunte ; `into_iter()` consomme.
9. Objet : état avec `struct`, méthodes avec `impl`, polymorphisme avec `trait`, réutilisation d'état par composition.
10. Threading : choisir d'abord entre transfert, channel et état partagé.
11. `Arc` partage l'ownership ; `Mutex`/`RwLock` protègent l'accès ; `thread::scope` permet l'emprunt borné.
12. Quand Rust refuse : demander d'abord **quel invariant est protégé ?**